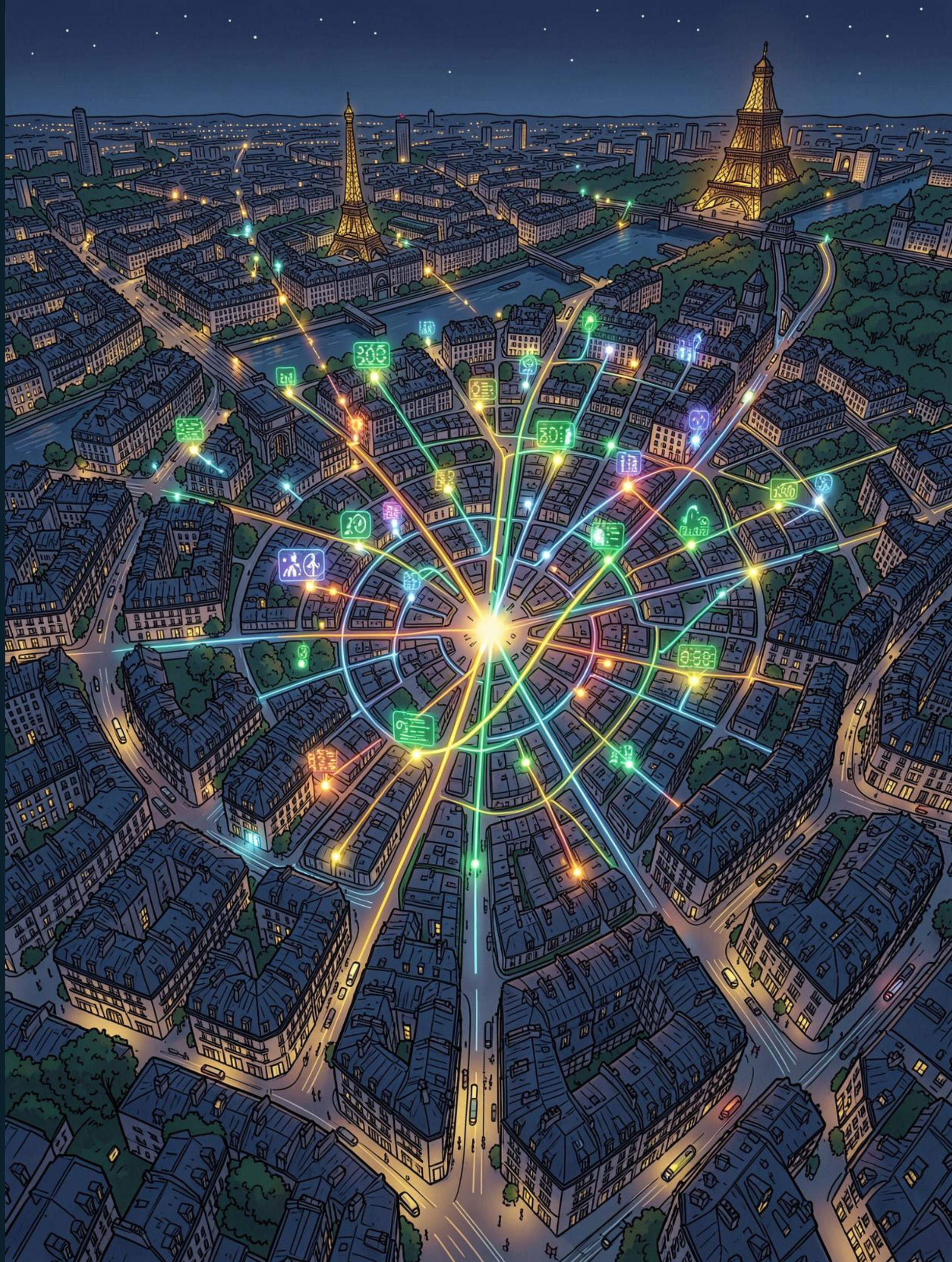
Holiday Itinerary Data Platform

-> From DATAtourisme raw data to an interactive Streamlit itinerary dashboard

-> A complete ETL + API + visualization stack

Presented by:

Hassan Salam B.
Niloufar Najafian



The problem we solved

Why

Tourism data is rich, but raw feeds are not directly usable for itinerary planning

Raw feed

Nested JSON, inconsistent field shapes, multilingual values, changing source files

Planning need

Users need city-level exploration, categories, maps, and day-by-day suggestions

Engineering need

Repeatable ingestion, incremental processing, and clear deployment packaging

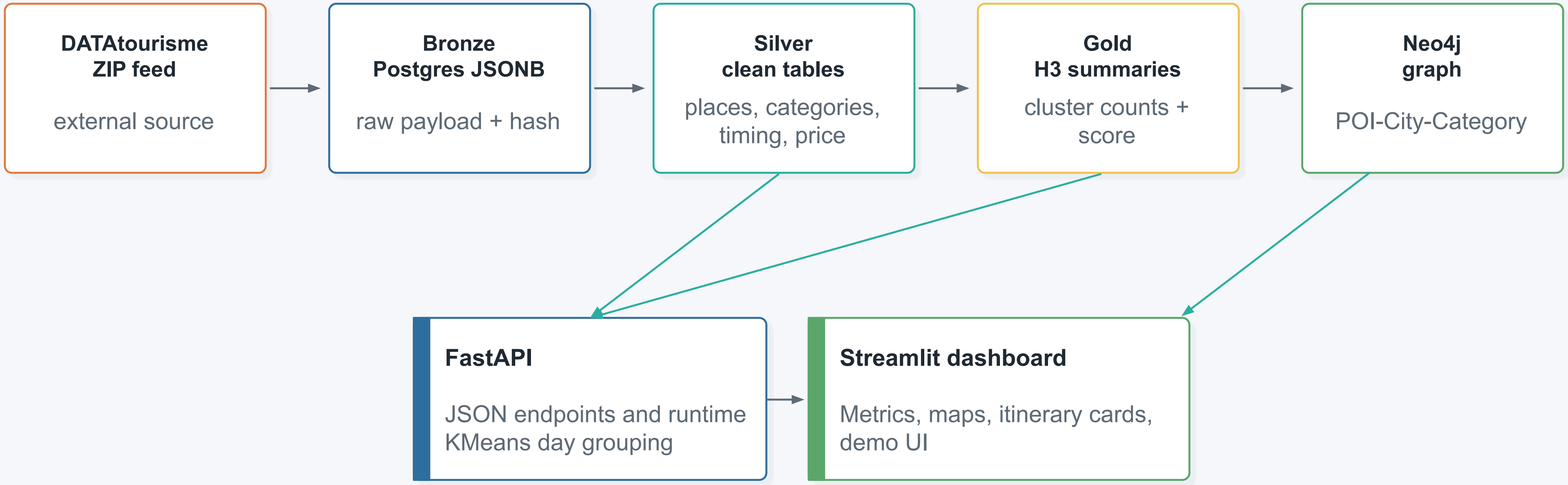
Our outcome

A Dockerized ETL + API + dashboard stack that converts raw POIs into a demoable itinerary application.

End-to-end architecture

Architecture

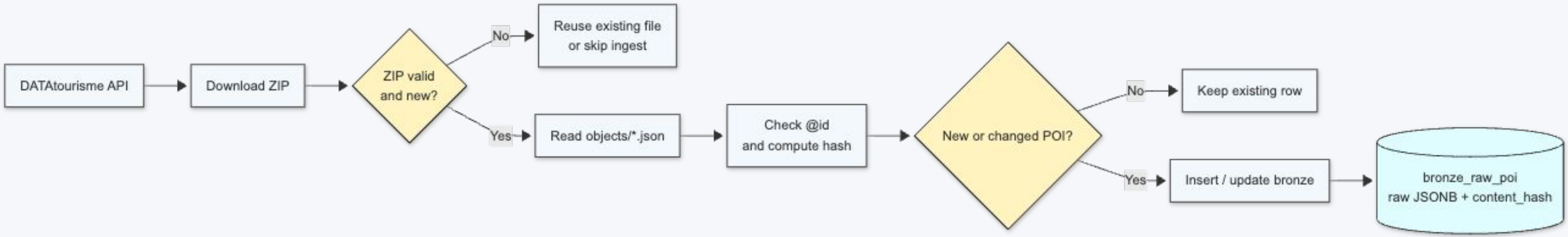
Two clean lanes: ETL builds trusted data; app services turn it into an itinerary experience



End-to-end architecture

Architecture

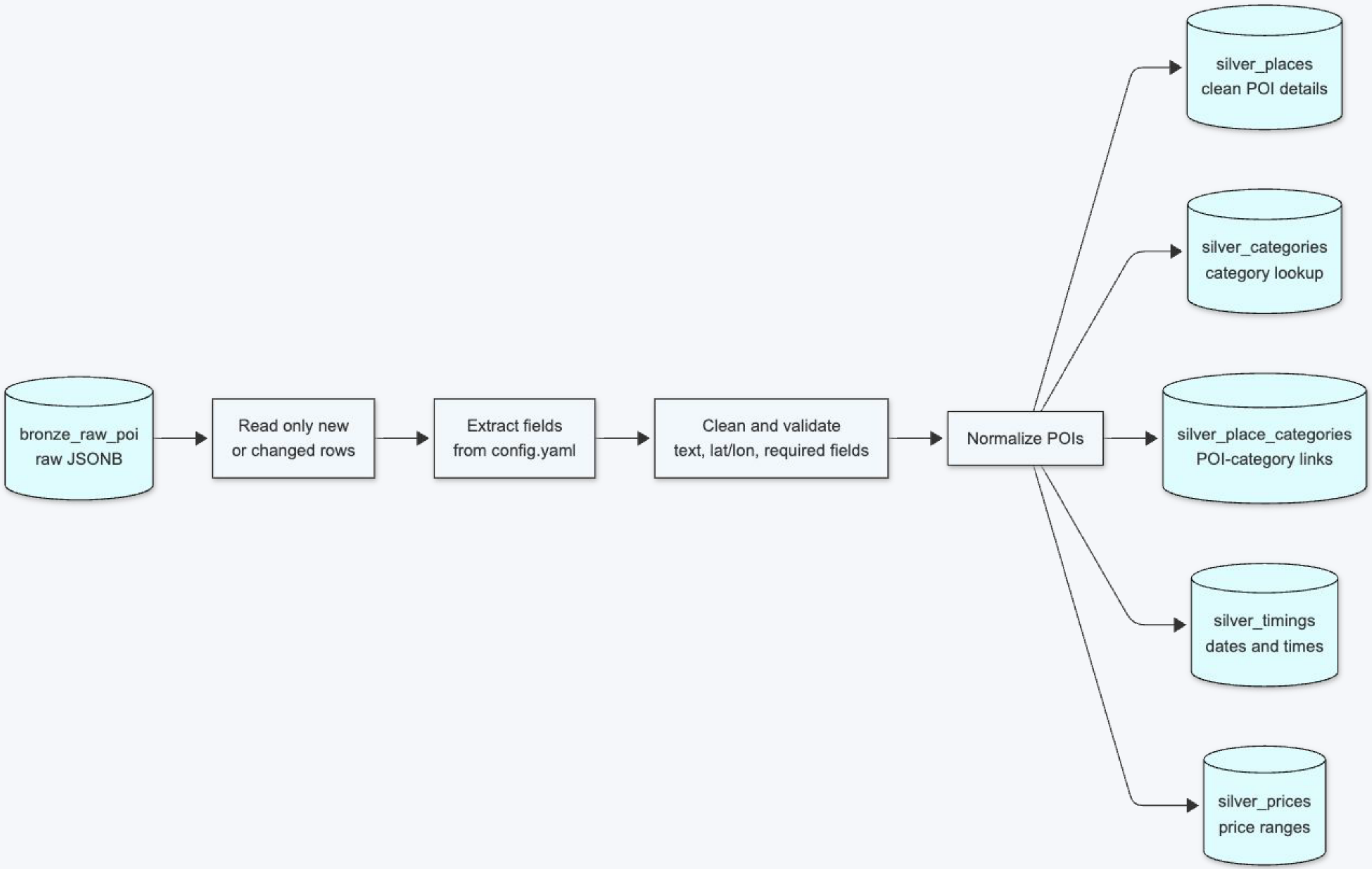
API ingestion



End-to-end architecture

Architecture

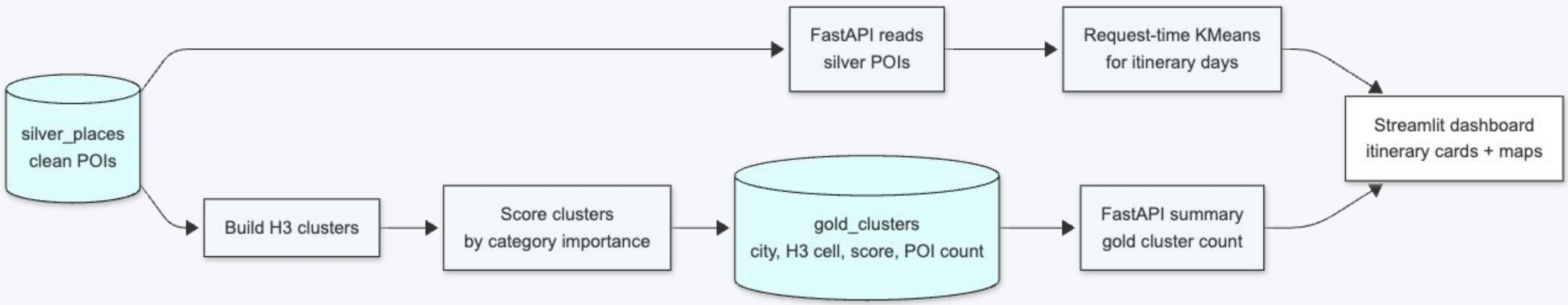
Bronze to silver database



End-to-end architecture

Architecture

Silver to gold database



Implemented stack and deferred options

Stack area

We used proven tools for local reproducibility and explainability; heavier options stayed out of scope for scale, setup, or time reasons

Orchestration	Python CLI, cron, and Docker Compose for repeatable ETL runs
Storage	Postgres JSONB for bronze, relational silver tables, and H3 gold summaries
Transformation + ML	SQL, Pandas chunks for cleaning; scikit-learn, KMeans for request-time day grouping
Graph layer	Neo4j for POI-City-Category relationships and related-place suggestions
API + UI	FastAPI as the JSON boundary; Streamlit for the interactive itinerary demo
Delivery	GitHub Actions checks plus Docker Hub image publishing for reusable releases

Itinerary generation: request-time intelligence

Algorithm

KMeans creates day areas; interests guide selection without over-filtering

1. User input
city, days, places/day, interests

→

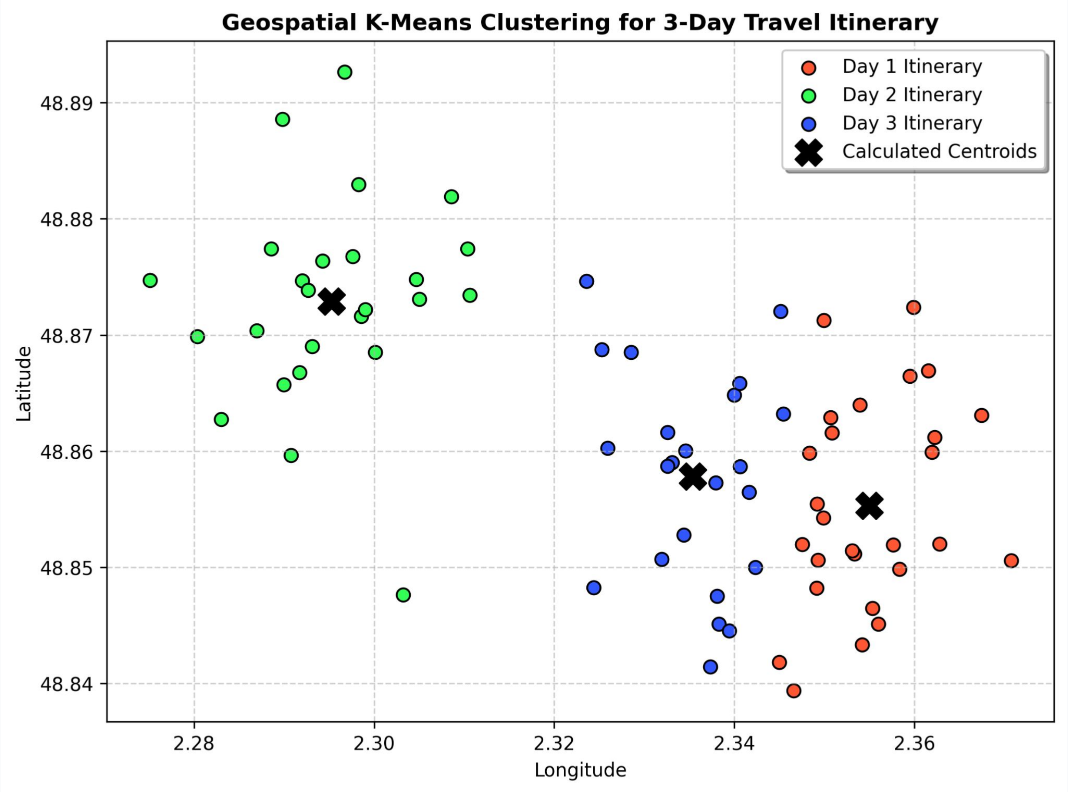
2. Silver POIs
read cleaned lat/lon + categories

→

3. KMeans
N clusters = requested days

→

4. Day plan
nearest to center + interest preference



Interests are preferences, not hard filters

Example: selecting Beach nudges the planner to include beach-related POIs, but each day is still filled with nearby varied stops.

Beach | Museum | Food

Holiday Itinerary API 1.0.0 OAS 3.1

/openapi.json

default

GET `/health` Health

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls `Accept` header.

Example Value | Schema

```
"string"
```

GET `/summary` Summary

GET `/cities` Cities

GET `/categories` Categories

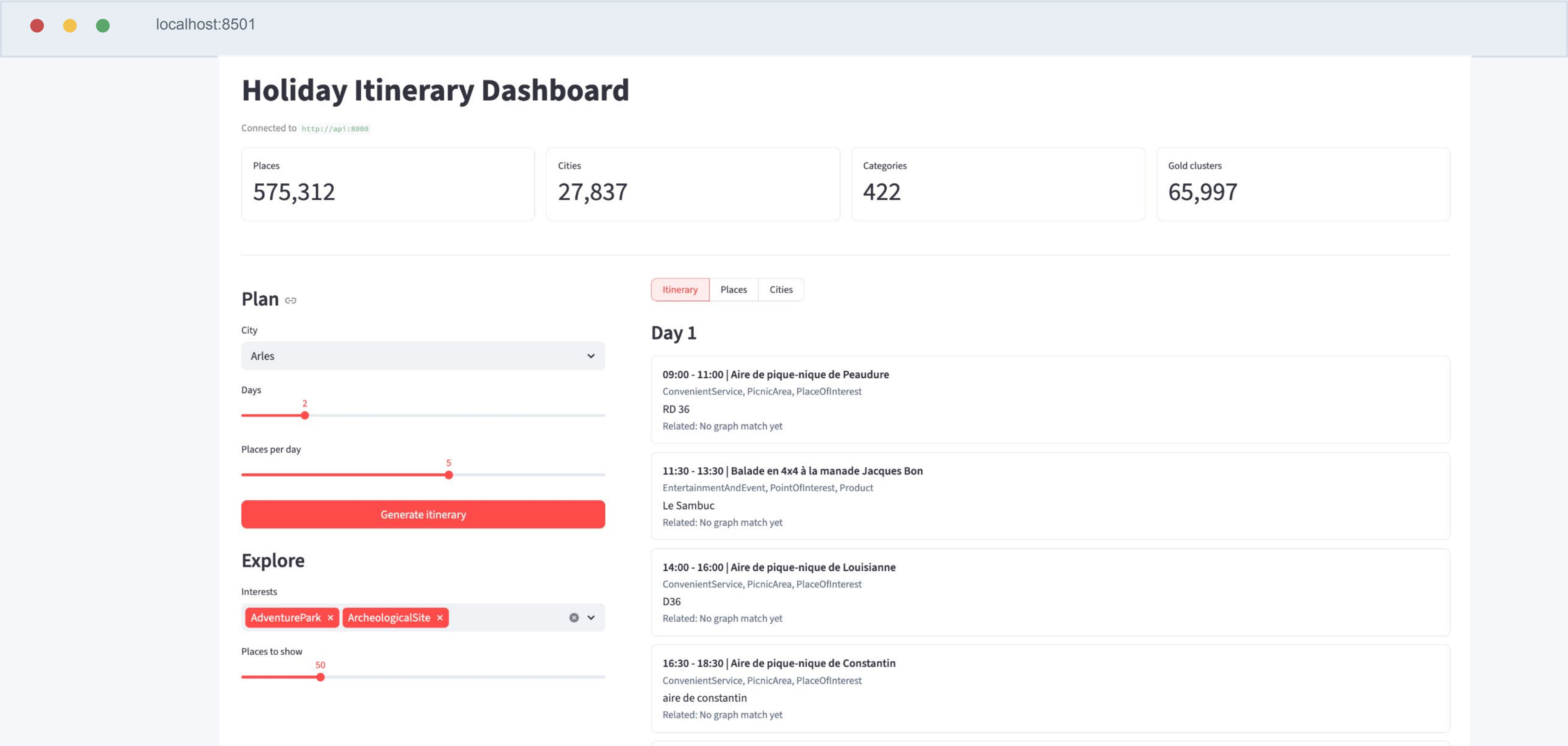
GET `/places` Places

POST `/generate-itinerary` Generate Itinerary

Streamlit demo walkthrough

Demo

The dashboard turns backend services into an instructor-friendly product demo



Streamlit demo walkthrough

Demo

The dashboard turns backend services into an instructor-friendly product demo

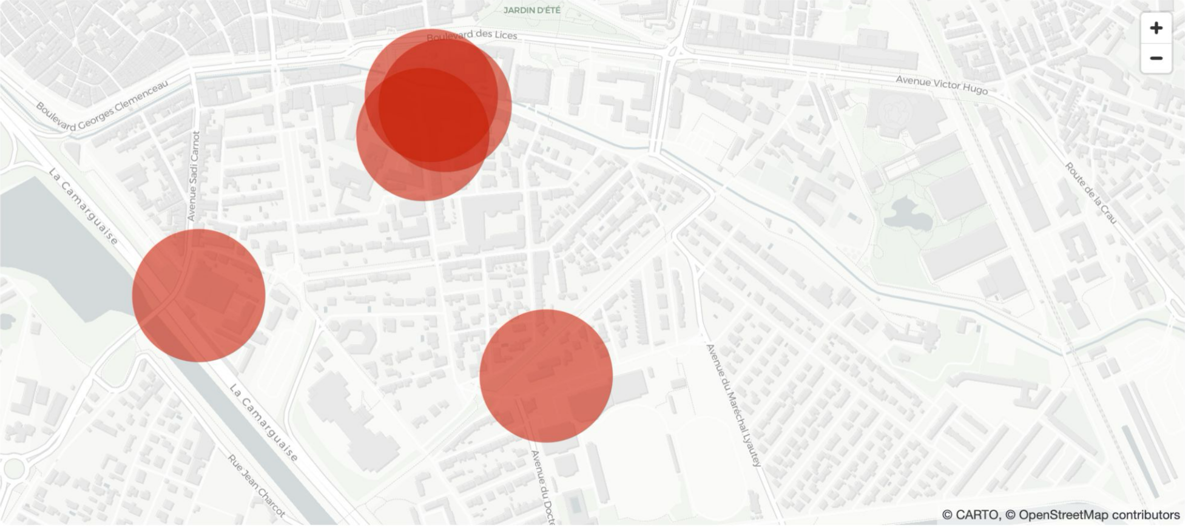
localhost:8501

Related: No graph match yet

14:00 - 16:00 | Arles Plaza
Accommodation, Hotel, HotelTrade
45 Avenue Sadi Carnot
Related: No graph match yet

16:30 - 18:30 | Découverte de la Camargue en 4X4 avec Camargue Alpilles Safari
EntertainmentAndEvent, PointOfInterest, Product
1 Boulevard Emile Fassin
Related: No graph match yet

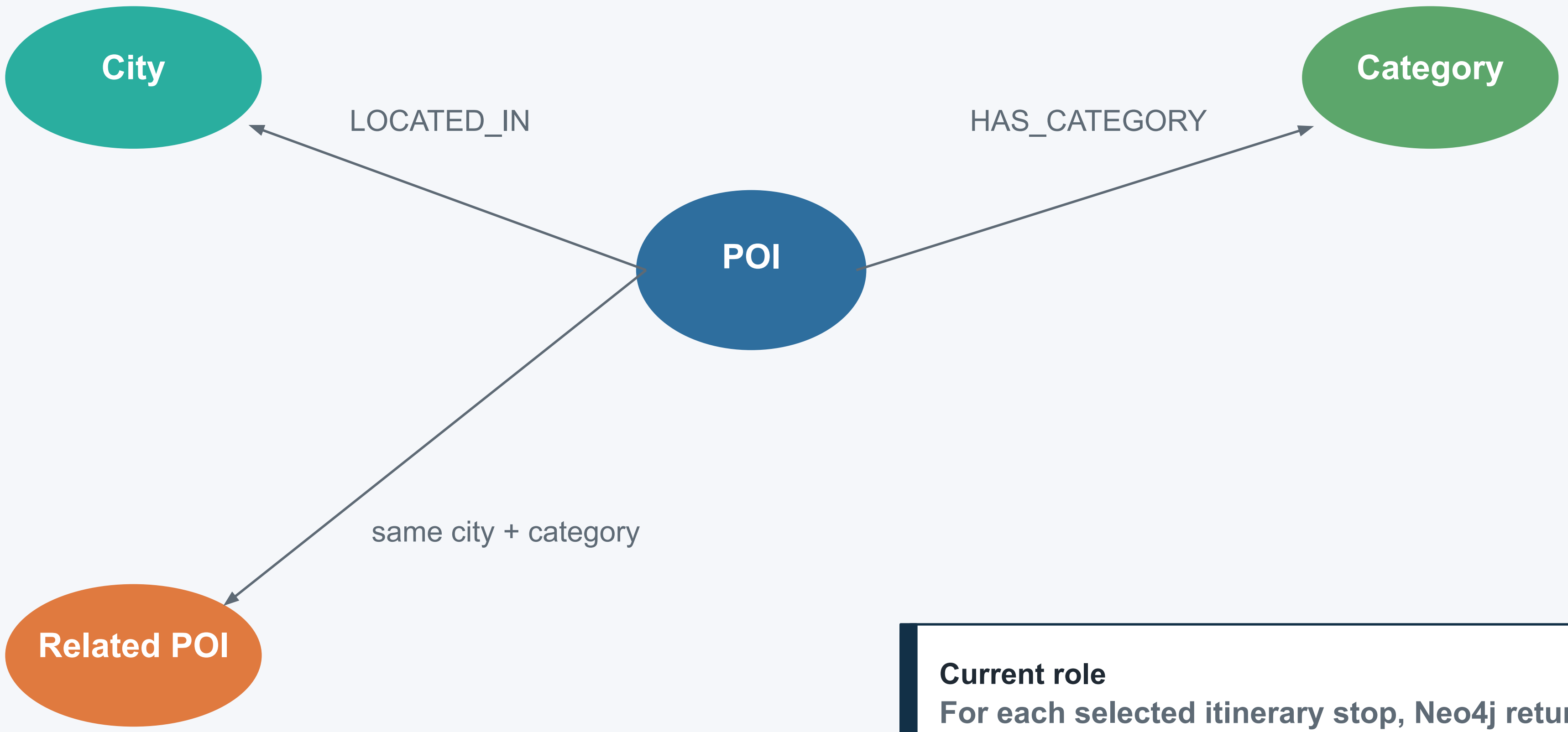
19:00 - 21:00 | Camargue Alpilles Safari
ActivityProvider, PlaceOfInterest, PointOfInterest
1, rue Emile Fassin
Related: No graph match yet



Where Neo4j fits

Graph

Neo4j adds relationship-aware suggestions; it does not choose the map or the day clusters



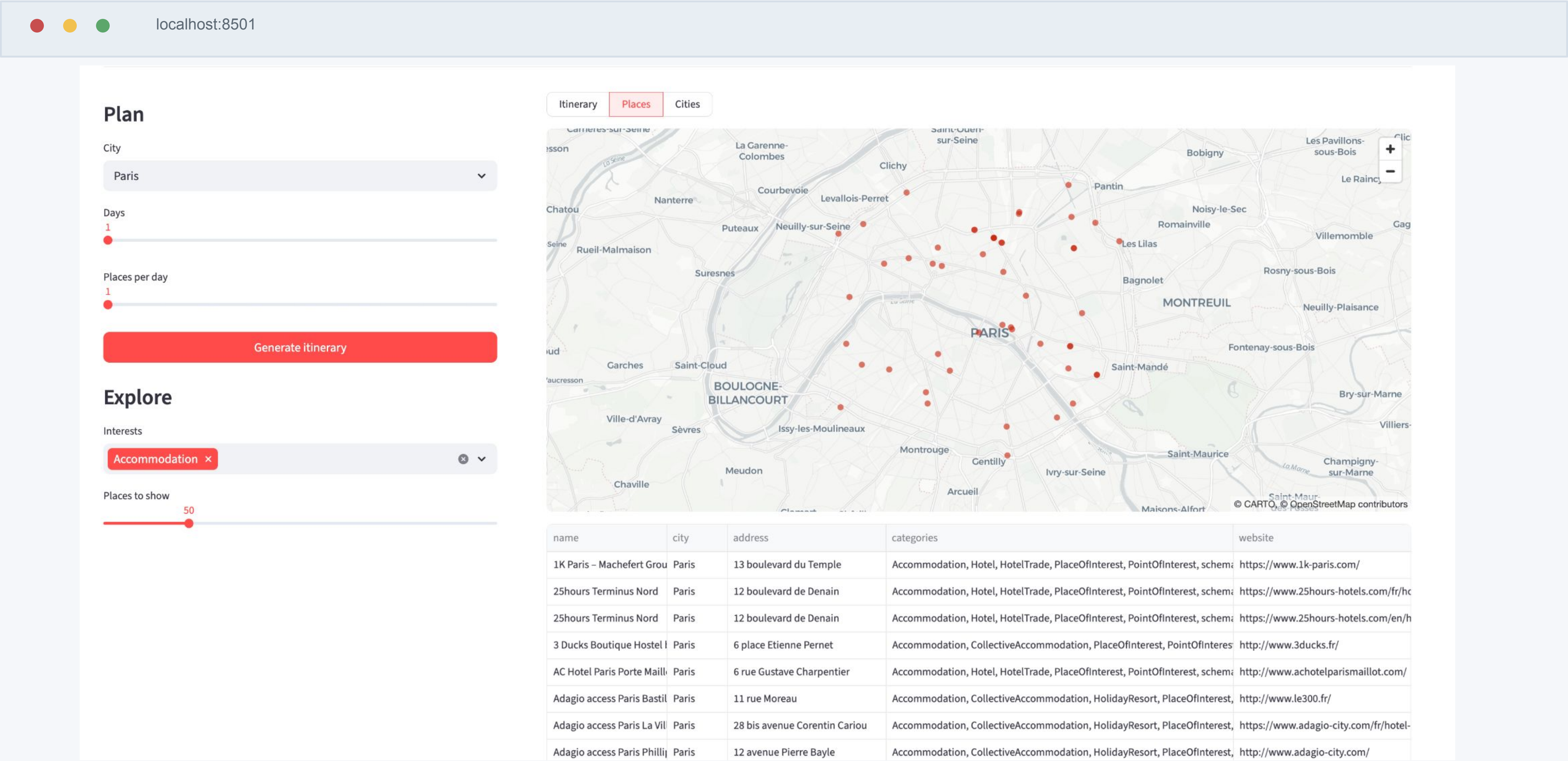
Current role

For each selected itinerary stop, Neo4j returns related POIs to show under the card.

Streamlit demo walkthrough

Demo

The dashboard turns backend services into an instructor-friendly product demo



CI/CD and distribution

GitHub Actions checks the project and publishes a reusable Docker image

Push / PR
dev or main

→

Checks
compile src
compose config
Docker build

→

Publish
Docker Hub image
if secrets exist

→

Run locally
release Compose
pull image

Docker Hub is not hosting

It stores a versioned app image. A machine or server still pulls and runs the containers.

Distribution method

Users can run `distribution/compose.release.yml` with an `.env` and the published image.

Engineering decisions and known limits

Evaluation

We optimized for reproducibility and clear boundaries; next steps improve route intelligence

Good decisions

- Incremental ETL with hashes and sync timestamps
- Silver layer as trusted source of detailed POIs
- FastAPI boundary between UI and data services
- Docker Compose for reproducible demos

Next improvements

- Real route optimization or travel-time API
- Better ranking from ratings, opening hours, prices
- Hosted demo environment, not only local Compose
- Monitoring ETL status in the dashboard

What we delivered

Close

A working, explainable data platform with a demoable itinerary application

4	2	1	CI
data layers	app services	Dockerized stack	validated delivery

Tourism data is rich — but raw feeds are not directly usable for itinerary planning. We built a pipeline that bridges the gap between messy source data and a clean, demoable product.

Raw Feed Reality

- Nested JSON with inconsistent field shapes
- Multilingual values across sources
- Changing source file structures

What Users & Engineering Needed

- City-level exploration with categories and maps
- Day-by-day itinerary suggestions
- Repeatable ingestion and incremental processing
- Clear, reproducible deployment packaging

Our outcome: A Dockerized ETL + API + dashboard stack that converts raw POIs into a demoable itinerary application.